

LAPORAN JOBSHEET PEMROGRAMAN BERBASIS FRAMEWORK



Nama : Yan Daffa Putra Liandhie

NIM: 2341720142

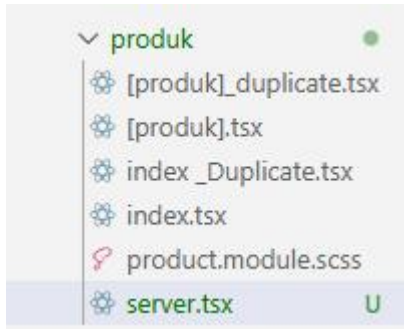
Kelas: TI 3E

POLITEKNIK NEGERI MALANG TAHUN 2026

✂ C. Langkah Praktikum

◆ Bagian 1 – Setup Halaman SSR

1. Buat file baru pada pages/products/server.tsx



2. Modifikasi file server.tsx :

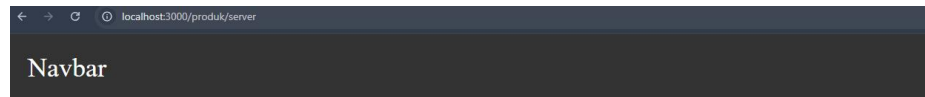
```
8.praktikum-Server_side rendering > my-app > src > pages > produk > server.tsx > default
1  import TampilanProduk from "../views/product";
2
3  Windsurf: Refactor | Explain | Generate JSDoc | X
4  const halamanProdukServer = () => {
5      return (
6          <div>
7              <h1>Halaman Produk Server</h1>
8              <TampilanProduk products={} />
9          </div>
10     );
11 };
12 export default halamanProdukServer;
```

```

1  import TampilanProduk from "../views/produk";
2
3  Windsurf: Refactor | Explain | Generate JSDoc | ✕
4  const halamanProdukServer = () => {
5    return (
6      <div>
7        <h1>Halaman Produk Server</h1>
8        <TampilanProduk products=[] />
9      </div>
10    );
11  }
12  export default halamanProdukServer;

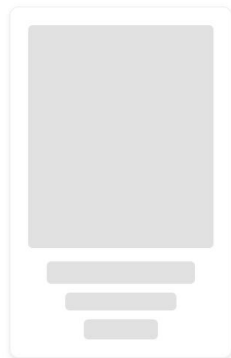
```

3. Jalankan browser : <http://localhost:3000/produk/server>



Halaman Produk Server

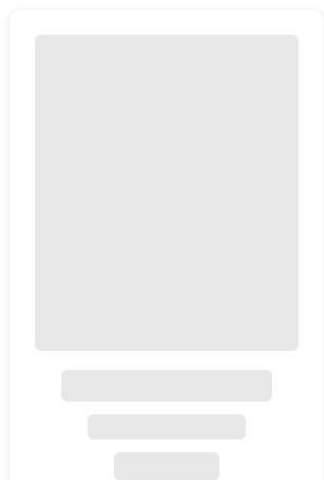
Daftar Produk



Navbar

Halaman Produk Server

Daftar Produk



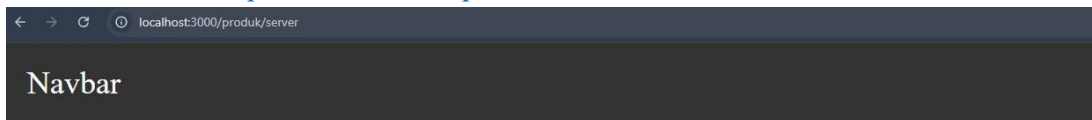
8.praktikum-Server_side rendering > my-app > src > pages > produk > server.tsx > getServerSideProps

```
1 import TampilanProduk from "../views/product";
2
3 type ProductType = {
4   id: string;
5   name: string;
6   price: number;
7   image: string;
8   category: string;
9 };
10
11
12 const halamanProdukServer = (props: {products: ProductType[]}) => {
13   const { products } = props;
14   return (
15     <div>
16       <h1>Halaman Produk Server</h1>
17       <TampilanProduk products={products} />
18     </div>
19   );
20 };
21
22 export default halamanProdukServer;
23
24 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
25 export async function getServerSideProps() {
26   const res = await fetch("http://localhost:3000/api/produk");
27   const response = await res.json();
28   // console.log("Data produk yang diambil dari API:", response);
29   return {
30     props: {
31       products: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
32     },
33   };
34 }
```

}

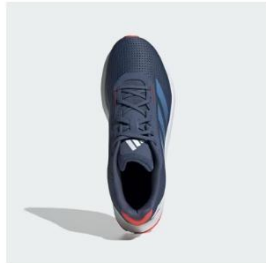
```
src / pages > produk > server.tsx > export default
1 import TampilanProduk from "../views/produk";
2
3 type ProductType = {
4   id: string;
5   name: string;
6   price: number;
7   image: string;
8   category: string;
9 };
10
11 const halamanProdukServer = (props: { products: ProductType[] }) => {
12   const { products } = props;
13   return (
14     <div>
15       <h1>Halaman Produk Server</h1>
16       <TampilanProduk products={products} />
17     </div>
18   );
19 };
20
21 export default halamanProdukServer;
22
23 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses,
24 // dan akan mengambil data produk dari API sebelum merender halaman.
25 export async function getServerSideProps() {
26   const res = await fetch("http://localhost:3000/api/produk");
27   const response = await res.json();
28   // console.log("Data produk yang diambil dari API:", response);
29   return {
30     props: {
31       products: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
32     },
33   };
34 };
35 }
```

- Jalankan browser <http://localhost:3000/produk/server>



Halaman Produk Server

Daftar Produk



Sepatu Duramo SL

Men's Shoes

Rp 900.000,00



SEPATU SAMBA OG

Men's Shoes

Rp 2.000.000,00



Halaman Produk Server

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000



sepatu pria

sepatu

Rp 300,000



celana

celana

Rp 150,000



baju polo

pakaian

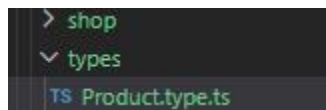
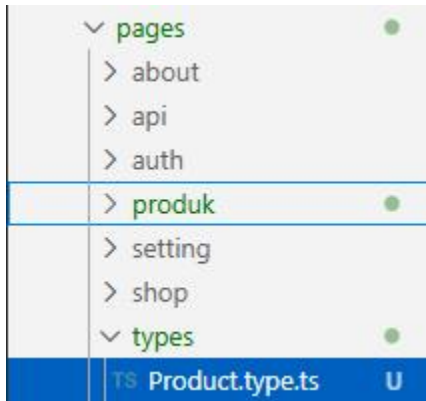
Rp 100,000

⚡ Catatan:

- Jika dijalankan maka skeleton tidak muncul yang akan muncul langsung pada produknya
- Harus menggunakan full URL
- Dipanggil setiap request halaman

◆ Bagian 3 – Refactor Type (produk type)

1. Buat folder types pada folder pages dan buat file Product.type.ts

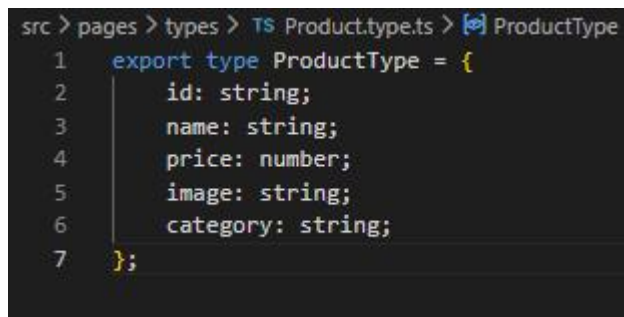


2. Modifikasi Product.type.ts

8.praktikum-Server_side rendering > my-app > src > pages > types > TS Product.type.ts > [🔗] ProductType

Windsurf: Refactor | Explain

```
1 export type ProductType = {
2   id: string;
3   name: string;
4   price: number;
5   image: string;
6   category: string;
7 };
```



3. Setelah membuat file Product.type.ts maka modifikasi pada file server.tsx menjadi

8.praktikum-Server_side rendering > my-app > src > pages > produk > server.tsx > [🔗] getServerSideProps

```
1 import TampilanProduk from "../views/product";
2 import { ProductType } from "../types/Product.type";
3
4
5
```

Windsurf: Refactor | Explain | Generate JSDoc | X

```
6 const halamanProdukServer = (props: {products: ProductType[]}) => {
7   const { products } = props;
8   return (
9     <div>
10       <h1>Halaman Produk Server</h1>
11       <TampilanProduk products={products} />
12     </div>
13   );
14 };
15 export default halamanProdukServer;
```

16 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.

Windsurf: Refactor | Explain | X

```
19 export async function getServerSideProps() {
20   const res = await fetch("http://localhost:3000/api/produk");
21   const response = await res.json();
22   // console.log("Data produk yang diambil dari API:", response);
23   return {
24     props: {
25       products: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
26     },
27   };
28 }
```

```

src > pages > produk > server.jsx > getServerSideProps
1 import TampilanProduk from "../views/produk";
2 import { ProductType } from "../types/Product.type";
3
4 const halamanProdukServer = (props: { products: ProductType[] }) => {
5   const { products } = props;
6   return (
7     <div>
8       <h1>Halaman Produk Server</h1>
9       <TampilanProduk products={products} />
10    </div>
11  );
12 };
13
14 export default halamanProdukServer;
15
16 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses,
17 // dan akan mengambil data produk dari API sebelum merender halaman.
18 export async function getServerSideProps() {
19   const res = await fetch("http://localhost:3000/api/produk");
20   const response = await res.json();
21   // console.log("Data produk yang diambil dari API:", response);
22
23   return {
24     props: {
25       products: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
26     },
27   };
28 }

```

◆ Bagian 4 – Uji Perbedaan SSR vs CSR

Uji 1 – Skeleton

- Buka halaman CSR
- Refresh
- Skeleton muncul
- Buka halaman SSR
- Refresh
- Skeleton tidak muncul

csr

Navbar

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000



sepatu pria

sepatu

Rp 300,000



celana

celana

Rp 150,000



baju polo

pakaian

Rp 100,000

Ssr

Navbar

Halaman Produk Server

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000



sepatu pria

sepatu

Rp 300,000



celana

celana

Rp 150,000



baju polo

pakaian

Rp 100,000

Uji 2 – Network Tab

1. Buka DevTools → Network → XHR
2. Refresh halaman CSR
→ Request API terlihat

Navbar

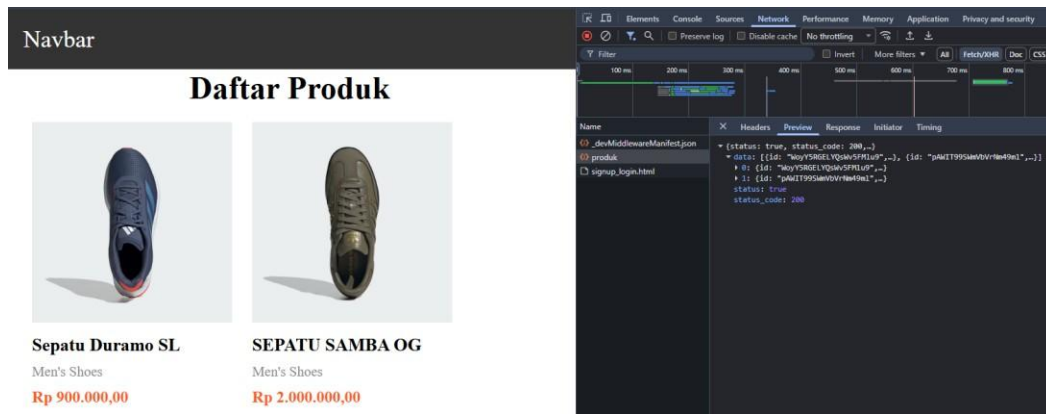
Daftar Produk

jaket
jaket
Rp 200,000

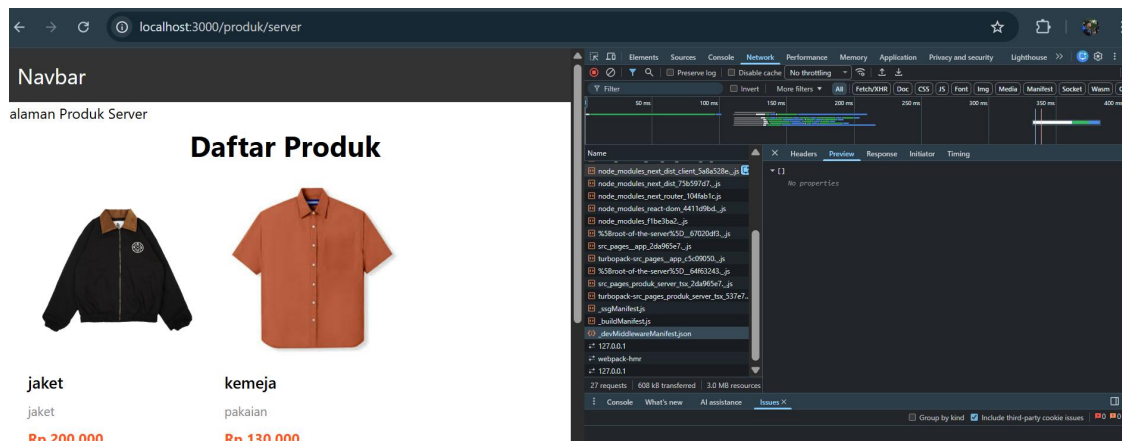
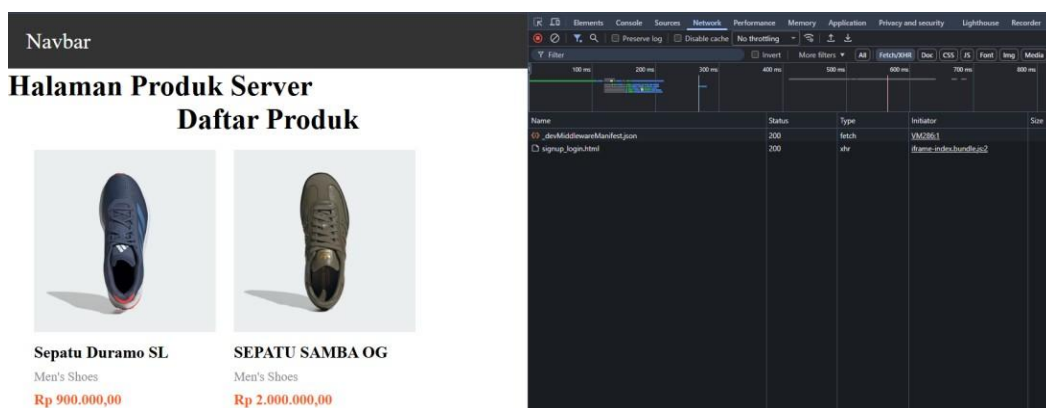
kemeja
pakaian
Rp 130,000

Network Tab:

- Filter: All
- Selected Request: produk
- Response: {status: true, status_code: 200, data: [{id: "R6ikw3cN9BU1U2dR8vG", category: "jaket", size: "XL", ...}], status: true, status_code: 200}

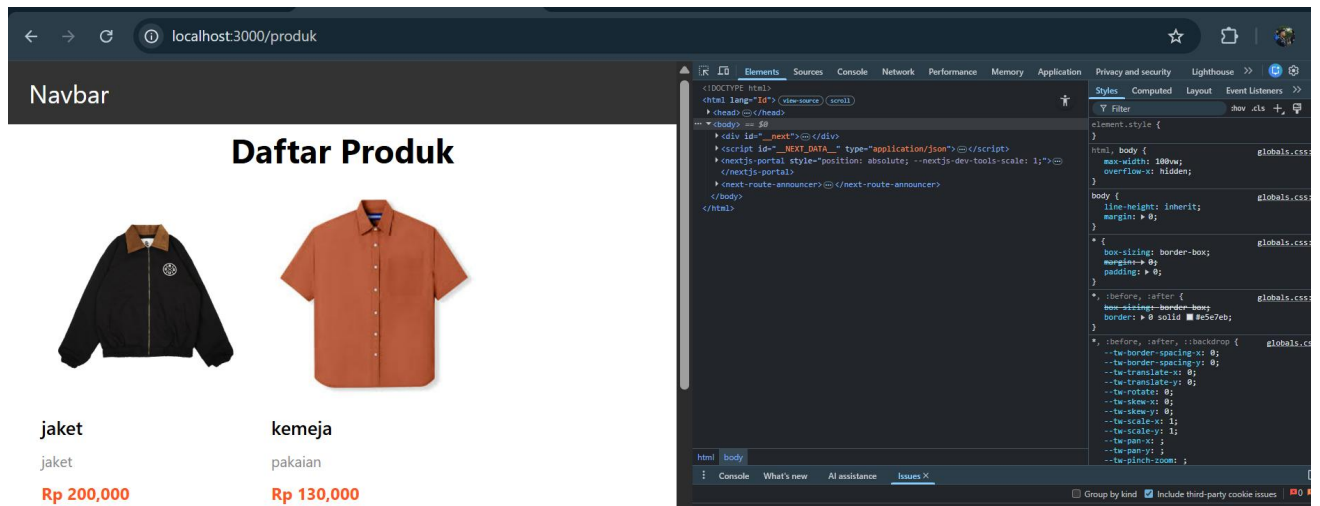


3. Refresh halaman SSR
→ Request API tidak terlihat

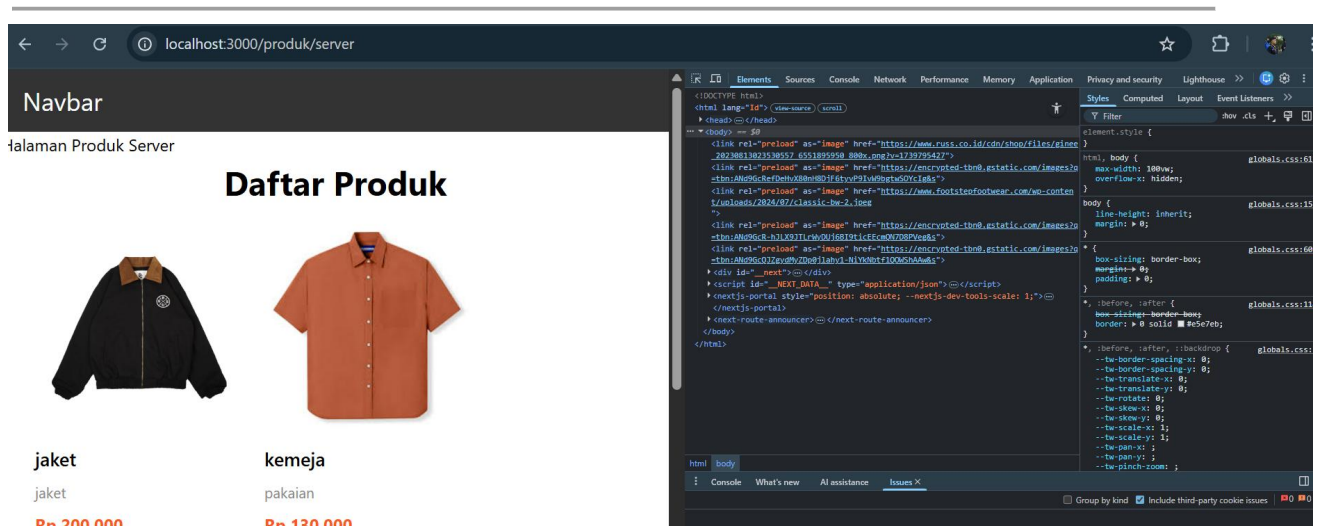


Uji 3 – Response HTML

- CSR: HTML awal kosong (berisi skeleton)



- SSR: HTML sudah berisi data produk lengkap

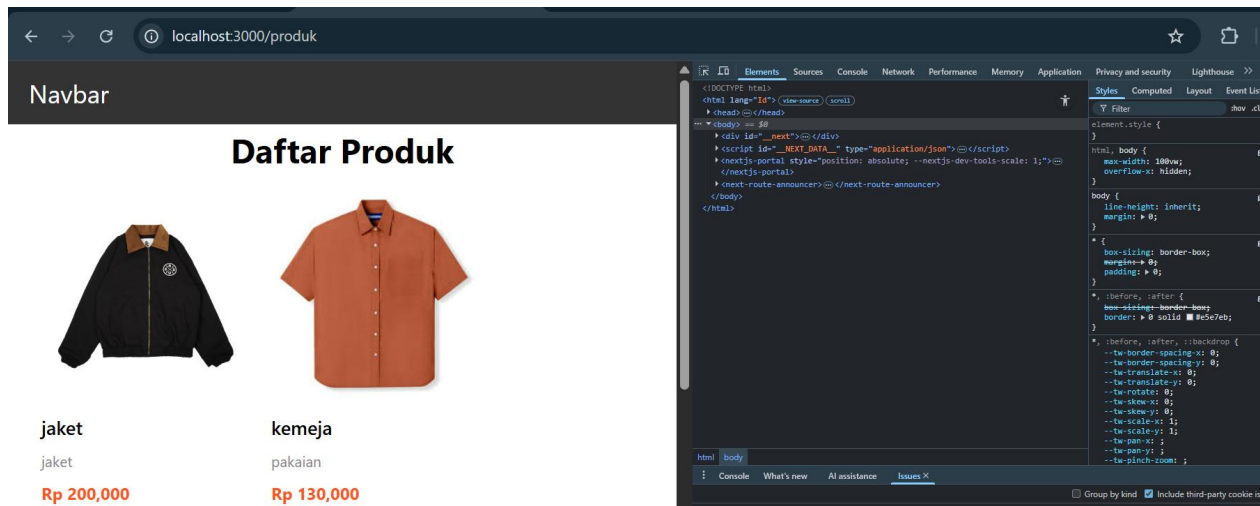


🎯 D. Tugas Praktikum

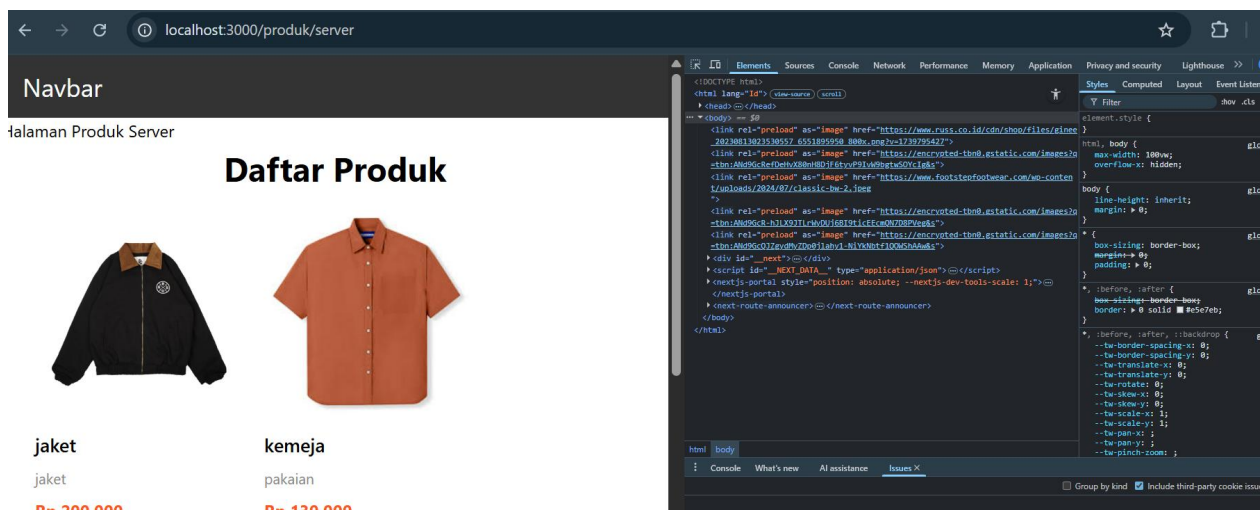
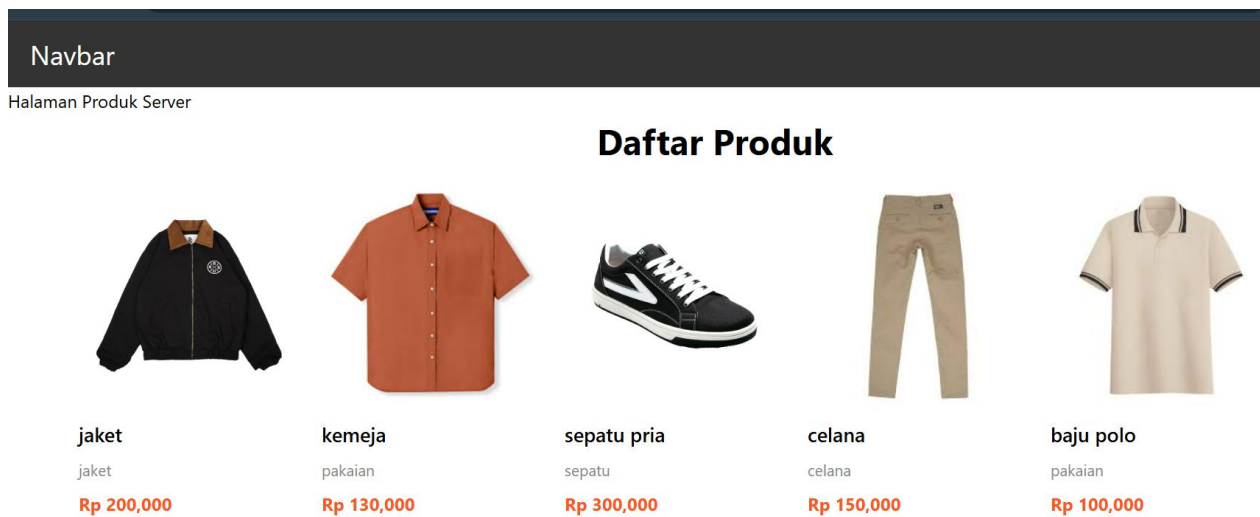
🔗 Tugas Individu

1. Buat 2 halaman:
 - o /products (CSR)

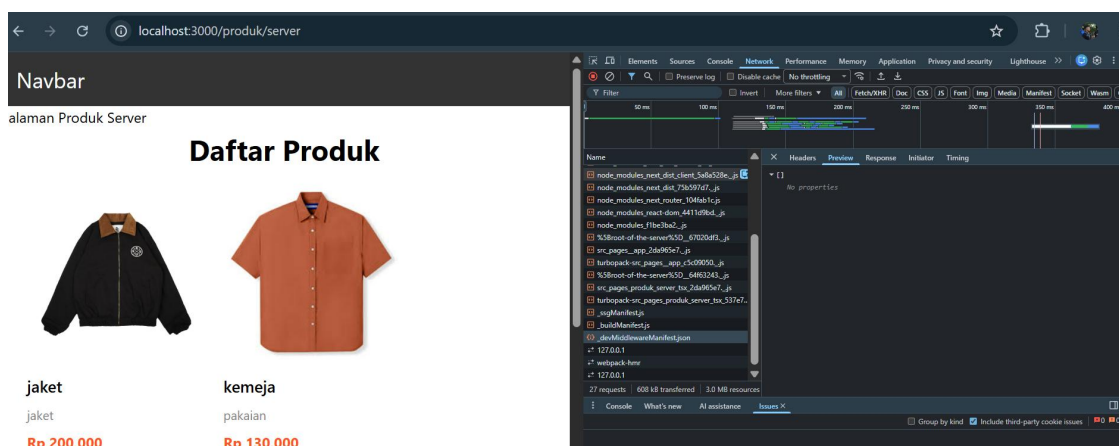
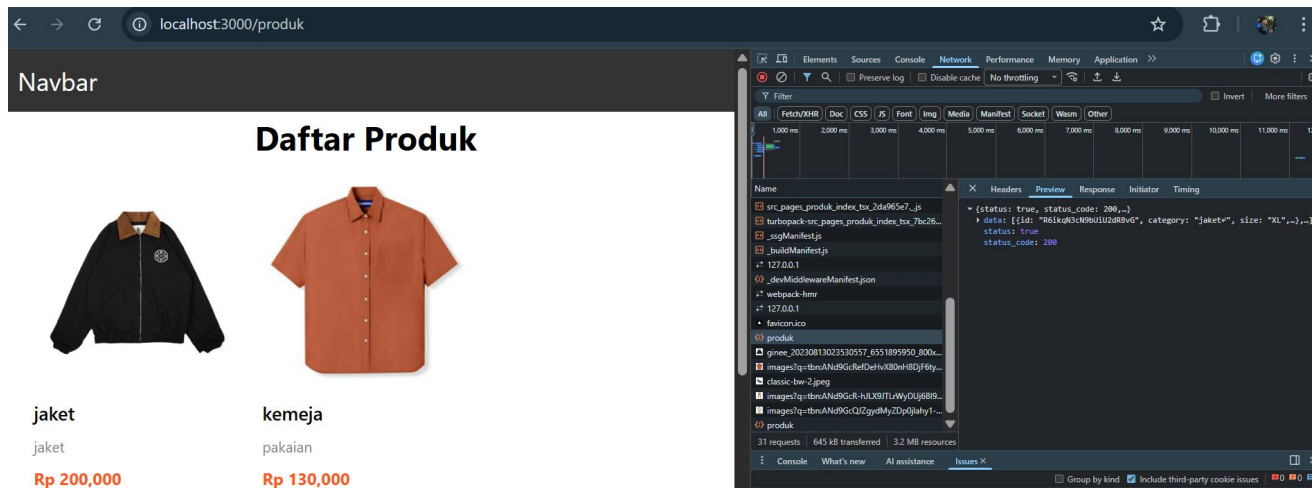




o /products/server (SSR)



o Perbedaan Network tab



Analisis:

Pada CSR, proses rendering dilakukan di browser. Server hanya mengirimkan file HTML dasar dan JavaScript, kemudian browser menjalankan JavaScript tersebut untuk mengambil data dan membangun tampilan halaman yang dapat mengakibatkan loading awal bisa lebih lambat dan kurang optimal untuk SEO. Sedangkan pada SSR, proses rendering dilakukan di server setiap kali ada permintaan dari pengguna, server langsung mengirimkan HTML yang sudah lengkap ke browser sehingga halaman bisa langsung ditampilkan. Metode ini lebih baik untuk SEO dan menampilkan data terbaru, tetapi membuat beban server lebih besar dibanding CSR

E. Studi Analisis

Jawab pertanyaan berikut:

1. Mengapa SSR lebih baik untuk SEO?

jawab : karena halaman sudah dirender menjadi HTML lengkap di server sebelum dikirim ke browser, sehingga dapat langsung membaca dan mengindeks seluruh konten tanpa harus menunggu eksekusi JavaScript seperti pada CSR.

2. Kapan sebaiknya menggunakan SSR?

Jawab : ketika aplikasi membutuhkan data yang selalu terbaru pada setiap request, membutuhkan optimasi SEO yang baik, atau ketika halaman harus cepat menampilkan konten awal seperti pada website berita, e-commerce, atau portal informasi

3. Apa kekurangan SSR dibanding CSR?

Jawab : beban server menjadi lebih besar karena setiap permintaan harus dirender ulang di server, serta waktu respons bisa meningkat jika proses pengambilan data atau rendering memakan waktu lama

4. Mengapa skeleton tidak muncul pada SSR?

Jawab :karena data sudah diambil dan halaman sudah dirender secara lengkap di server sebelum dikirim ke browser, jadi pengguna langsung menerima konten jadi tanpa perlu menunggu proses fetch data di sisi client